

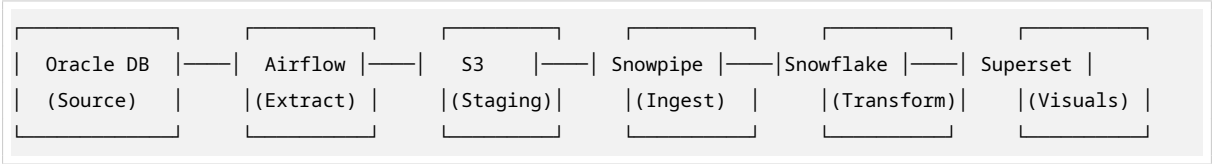
Bus Censor Data Pipeline — Project Documentation

Project Type: Data Engineering Portfolio Project
Platform: Arch Linux + Hyprland
Last Updated: 2026-07-30
Status: In Progress (Airflow initialization pending)

1. Project Overview

This project implements a modern data pipeline that extracts bus sensor/censor data from an Oracle transactional database, stages it in AWS S3, auto-ingests it into Snowflake via Snowpipe, transforms it, and visualizes it in Apache Superset.

Architecture Diagram



Pipeline Flow

- Oracle DB** — Raw bus sensor/censor transactional data
- Apache Airflow** — Orchestrates extraction, light transformation, and S3 loading
- AWS S3** — Cheap, durable data lake staging area
- Snowpipe** — Serverless auto-ingestion from S3 into Snowflake
- Snowflake** — Data warehouse with raw → clean → serve layers
- Apache Superset** — Dashboards and SQL Lab analytics

2. System Environment

Component	Spec
OS	Arch Linux with Hyprland (Wayland compositor)
CPU	Intel i7-13700H
GPU	NVIDIA RTX 4050 Max-Q (6GB VRAM)
RAM	16GB
Python	3.14.6
Package Manager	pipx 1.15.0

Active Docker Containers

Container	Image	Ports	Status
Oracle XE	gvenzl/ oracle-xe:21-slim	1521:1521	✅ Running (20+ min)
Portainer	portainer/ portainer-ce:latest	9000:9000	✅ Running
Odysseus AI	odysseus-odysseus	7000:7000	✅ Running
SearXNG	searxng/ searxng:2026.5.31	8080:8080	✅ Healthy
ChromaDB	chromadb/ chroma:latest	8100:8000	✅ Running
ntfy	binwiederhier/ntfy	8091:80	✅ Running

3. What Has Been Completed ✅

3.1 Oracle Database (Source Layer)

- **Installed:** Oracle XE 21c via Docker (gvenzl/oracle-xe:21-slim)
- **Status:** Container running, SQL*Plus verified (v21.3.0.0.0)
- **Port:** localhost:1521
- **Credentials:** SYSTEM/admin (default setup)

3.2 AWS S3 (Staging Layer)

- **Account:** AWS Free Tier active
- **Region:** ap-south-1
- **Bucket Created:** mkp-s3-data
- **AWS CLI:** Installed and configured with IAM Access Keys
- **Verification:** `aws s3 ls` successfully lists the bucket

3.3 Python Environment

- **pipx:** v1.15.0 installed and functional
- **Apache Airflow:** v3.3.0 installed via pipx (Python 3.14.6)
- **Boto3:** Injected into Airflow virtualenv (confirmed present)
- **cx_Oracle:** Planned for Oracle connectivity
- **pandas:** Planned for data transformation

3.4 Snowflake (Data Warehouse)

- **Web UI:** Accessible and functional
- **SnowSQL CLI:** Installed and configured
- **Status:** Ready for Snowpipe and stage configuration

3.5 Apache Superset (Visualization)

- **Status:** Installed and running
- **Connection:** Already connected to Snowflake (credit card verified)
- **Purpose:** Dashboards for bus censor analytics

4. Current Blockers / Pending Tasks 🕒

#	Issue	Status	Next Action
1	airflow command not found	🛑 Blocker	Fix PATH or locate pipx binary location
2	Airflow database not initialized	🕒 Pending	Run <code>airflow db init</code> after PATH fix
3	Airflow admin user not created	🕒 Pending	Create user after db init
4	Airflow webserver/scheduler not started	🕒 Pending	Start services in separate terminals
5	Oracle schema/tables not created	🕒 Pending	Create bus sensor tables in Oracle
6	S3 folder prefixes not created	🕒 Pending	Create <code>bus-censor/raw/</code> structure
7	Snowpipe not configured	🕒 Pending	Create storage integration, stage, and pipe
8	Airflow DAG not written	🕒 Pending	Write Oracle → S3 extraction DAG
9	Snowflake transformation layer	🕒 Pending	Raw → Clean → Serve tables/views
10	Superset dashboards	🕒 Pending	Build charts on curated Snowflake data

Blocker Detail: Airflow PATH Issue

Problem: airflow binary is not in \$PATH after pipx install.

Attempted:

```
ls ~/.local/bin/airflow
# Result: No such file or directory
```

Likely Cause: pipx may have installed the binary to a different location, or the venv apps path is not linked to ~/.local/bin.

Resolution Options:

1. Check pipx venv location: `pipx list`
2. Find actual binary: `find ~/.local/share/pipx -name "airflow" -type f`
3. Ensure ~/.local/bin is in PATH: `echo $PATH`
4. Reinstall with `--force` if binary is missing

5. Tech Stack Summary

Layer	Tool	License / Cost	Purpose
Source DB	Oracle XE 21c	Free (12GB limit)	Transactional bus sensor data
Orchestration	Apache Airflow 3.3.0	Apache 2.0 (Free)	DAG-based ETL orchestration
Object Storage	AWS S3	Free Tier (5GB)	Data lake staging
Auto-Ingestion	Snowpipe	Snowflake credits	Serverless S3 → Snowflake
Data Warehouse	Snowflake	Trial (\$400 credits)	OLAP, transformations
BI / Viz	Apache Superset	Apache 2.0 (Free)	Dashboards & SQL Lab
Container Runtime	Docker	Free	Oracle XE hosting

6. Project Directory Structure (Planned)

```
~/airflow/
├─ dags/
│   └─ bus_censor_pipeline.py      # Main extraction DAG
├─ logs/
├─ plugins/
└─ airflow.db                    # SQLite metadata (dev)

~/bus-censor-project/
├─ sql/
│   ├── oracle/
│   │   └─ create_tables.sql      # Bus sensor schema
│   └─ snowflake/
│       ├── 01_storage_integration.sql
│       ├── 02_external_stage.sql
│       ├── 03_snowpipe.sql
│       ├── 04_raw_tables.sql
│       ├── 05_clean_transforms.sql
│       └─ 06_serve_views.sql
├─ dbt/                          # (Optional future)
│   └─ models/
├─ data/
│   └─ sample_bus_data.parquet
└─ docs/
    └─ README.md                 # This file
```

7. Key Design Decisions

Why Real S3 over LocalStack?

- **Portfolio Value:** Recruiters prefer real AWS implementations
- **Free Tier:** 5GB S3 storage + 20,000 GET requests/month is sufficient

- **Skill Demonstration:** Shows actual IAM, bucket policy, and event notification knowledge
- **Snowpipe Integration:** Real S3 → SQS/SNS → Snowpipe is production-grade

Why Oracle XE in Docker?

- **Zero Cost:** Oracle Express Edition is free (12GB data limit)
- **Easy Teardown:** Containerized for clean project isolation
- **Realistic:** Many enterprises still run Oracle for transactional systems

Why pipx over system pip?

- **Isolation:** Each tool runs in its own venv (no dependency conflicts)
- **Arch Linux Friendly:** Aligns with Arch's minimal philosophy
- **Clean Upgrades:** Easy to uninstall/reinstall without breaking system Python

8. Next Steps (Priority Order)

Immediate (Fix Blocker)

1. Locate Airflow binary: `find ~/.local/share/pipx -name "airflow"`
2. Add correct path to `~/.bashrc` or `~/.zshrc`
3. Initialize Airflow: `airflow db init`
4. Create admin user and start webserver + scheduler

Phase 1: Data Generation & Extraction

5. Create bus sensor tables in Oracle (sample schema with timestamps, sensor IDs, readings)
6. Generate/mock sample data
7. Write Airflow DAG using `OracleHook` → `S3Hook`
8. Test DAG run and verify Parquet files land in `s3://mkp-s3-data/bus-censor/raw/`

Phase 2: Snowflake Ingestion

9. Create Storage Integration for S3 in Snowflake
10. Create External Stage pointing to S3 bucket
11. Create Snowpipe with `AUTO_INGEST = TRUE`
12. Configure S3 event notifications (SQS) to trigger Snowpipe
13. Verify data lands in Snowflake raw table

Phase 3: Transformation & Serving

14. Build raw → clean transformation SQL (deduplication, type casting)
15. Build clean → serve views (aggregations, star schema)
16. Test query performance

Phase 4: Visualization

17. Connect Superset to Snowflake serve schema
18. Build dashboards: sensor health, temporal trends, anomaly detection
19. Configure caching and row-level security if needed

9. Cost Projection (Learning vs Production)

Scenario	Estimated Cost	Notes
Learning (Current)	\$0	AWS Free Tier + Snowflake Trial + Open Source tools
Small Production	\$50–200/month	S3 storage + Snowflake compute (XS warehouse)
Medium Production	\$500–2000/month	Larger Snowflake warehouse + more data volume

Current Stack Cost: \$0 — All tools are free/open-source or within free tier limits.

10. Commands Reference

Docker

```
docker ps                # List running containers
docker exec -it oracle sqlplus -V # Verify Oracle
docker logs oracle -f     # Watch Oracle logs
```

AWS S3

```
aws s3 ls                # List buckets
aws s3 ls s3://mkp-s3-data # List bucket contents
aws s3 mb s3://mkp-s3-data/bus-censor/raw/ # Create prefix
```

pipx

```
pipx list                # List installed apps
pipx inject apache-airflow boto3 # Add package to Airflow venv
pipx reinstall apache-airflow    # Force reinstall
```

Airflow (Once PATH Fixed)

```
export AIRFLOW_HOME=~/.airflow
airflow db init
airflow users create --username admin --password admin --firstname Admin --lastname User --role Admin
↵ --email admin@local.com
airflow webserver --port 8080    # Terminal 1
airflow scheduler                # Terminal 2
```

11. Notes & Learnings

- **Arch Linux Python:** System Python 3.14.6 is very new; some packages may need `--pre` flags or compatibility patches.
- **pipx vs pip:** `pip` command is not available system-wide; `python -m pip` also fails. `pipx` is the correct path for user-level tool installation.
- **Airflow 3.3.0:** This is a very recent version (released 2026). Documentation online may still reference Airflow 2.x patterns. Be prepared for API differences.
- **Oracle InstantClient:** Not needed since using Dockerized Oracle XE with `gvenzl` image (includes client libraries).
- **AWS Region:** `ap-south-1` chosen for low latency from user's location (India timezone +05:30).

Document generated for portfolio and reference purposes. Update as project progresses.